

Rapport Projet **GoatUdder**  
Pipeline DevOps & Architecture Logicielle

GoatSoftware  
MENON Valentin, MASSINOND Matéo

21 juin 2026

# Table des matières

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                        | <b>3</b>  |
| 1.1      | Contexte . . . . .                                         | 3         |
| 1.2      | Objectifs DevOps . . . . .                                 | 3         |
| <b>2</b> | <b>Architecture Logicielle</b>                             | <b>4</b>  |
| 2.1      | Architecture globale . . . . .                             | 4         |
| 2.2      | Architecture en couches (par service) . . . . .            | 5         |
| 2.2.1    | Goat Service . . . . .                                     | 5         |
| 2.2.2    | Rental Service . . . . .                                   | 5         |
| 2.3      | Architecture Docker . . . . .                              | 5         |
| <b>3</b> | <b>Pipeline CI (Continuous Integration)</b>                | <b>6</b>  |
| 3.1      | Configuration GitHub Actions . . . . .                     | 6         |
| 3.2      | Étapes du pipeline . . . . .                               | 7         |
| <b>4</b> | <b>Rapport de Tests</b>                                    | <b>8</b>  |
| 4.1      | Tests unitaires (couche Service) . . . . .                 | 8         |
| 4.1.1    | Goat Service . . . . .                                     | 8         |
| 4.1.2    | Rental Service . . . . .                                   | 8         |
| 4.2      | Tests des Controllers (Mocks web avec Supertest) . . . . . | 8         |
| 4.2.1    | Goat Controller . . . . .                                  | 8         |
| 4.2.2    | Rental Controller . . . . .                                | 9         |
| 4.3      | Tests de la couche Data (Repository) . . . . .             | 9         |
| <b>5</b> | <b>Rapport de Couverture de Code</b>                       | <b>10</b> |
| 5.1      | Seuils de couverture . . . . .                             | 10        |
| 5.2      | Résultats de couverture . . . . .                          | 10        |
| <b>6</b> | <b>Rapport de Qualité</b>                                  | <b>11</b> |
| 6.1      | SonarQube . . . . .                                        | 11        |
| 6.2      | ESLint . . . . .                                           | 12        |

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| 6.3      | Code coverage de JEST et tests unitaires . . . . . | 12        |
| <b>7</b> | <b>Captures d'écran des Google Labs</b>            | <b>13</b> |
| <b>8</b> | <b>Accès aux Ressources</b>                        | <b>15</b> |
| 8.1      | Dépôt Git (GitHub) . . . . .                       | 15        |
| 8.2      | Harbor Registry (Images Docker) . . . . .          | 15        |
| <b>9</b> | <b>Conclusion</b>                                  | <b>16</b> |

# Chapitre 1

## Introduction

### 1.1 Contexte

Le projet **GoatUdder** est une plateforme web minimaliste de location de pis de chèvres. L'utilisateur peut louer un ou plusieurs pis pour une durée limitée et obtenir du lait de chèvre selon un modèle de location simple.

### 1.2 Objectifs DevOps

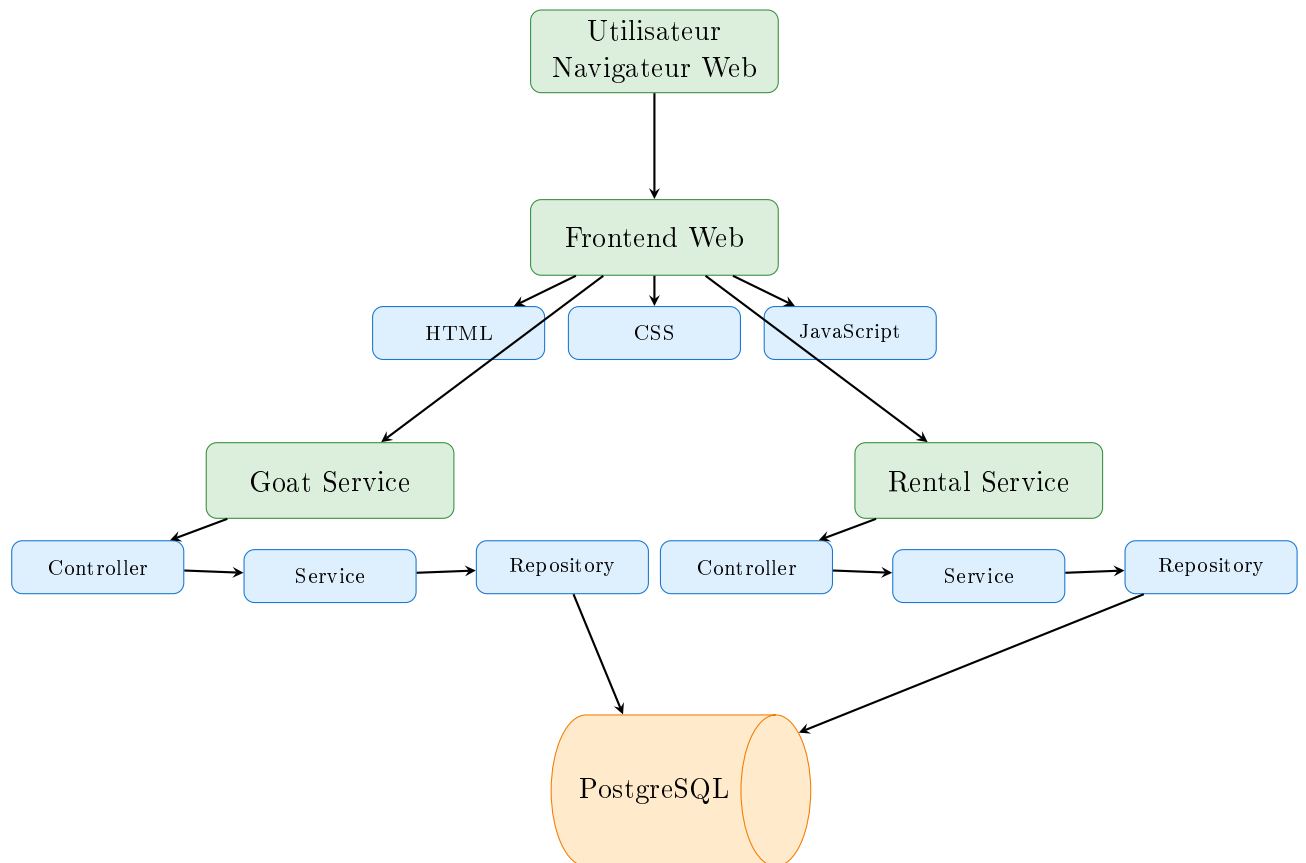
Développer une application en respectant les règles du DevOps :

- Dépôt Git structuré
- Pipeline CI (Continuous Integration) avec GitHub Actions
- Architecture logicielle en couches (Data, Services, Controller)
- Conteneurisation Docker de tous les services
- Tests de toutes les couches (unitaires, mocks web)
- Bonne couverture de code ( $\geq 80\%$ )
- Qualité logicielle élevée (SonarQube, ESLint)

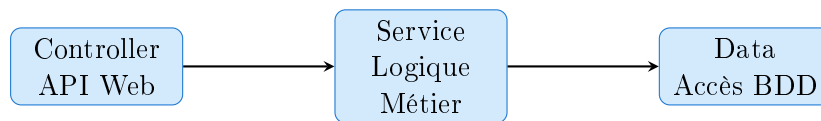
# Chapitre 2

## Architecture Logicielle

### 2.1 Architecture globale



## 2.2 Architecture en couches (par service)



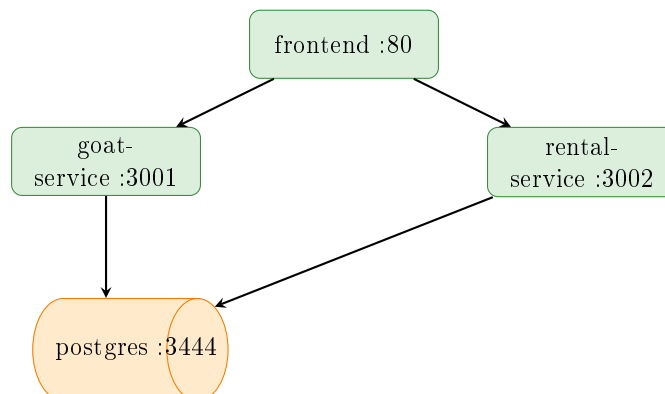
### 2.2.1 Goat Service

- **Controller** : `goat-controller.js` — Gestion des endpoints API pour les chèvres
- **Service** : `goat-service.js` — Logique métier (disponibilité, état des chèvres)
- **Data** : `goat-repository.js` — Accès PostgreSQL pour les données de chèvres

### 2.2.2 Rental Service

- **Controller** : `rental-controller.js` — Gestion des endpoints API pour les locations
- **Service** : `rental-service.js` — Logique métier (création, suivi, complétion des locations)
- **Data** : `rental-repository.js` — Accès PostgreSQL pour les données de locations

## 2.3 Architecture Docker



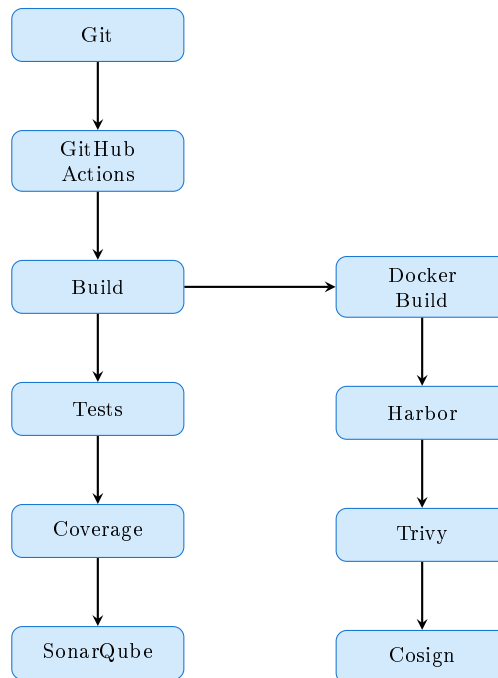
# Chapitre 3

## Pipeline CI (Continuous Integration)

### 3.1 Configuration GitHub Actions

Le pipeline CI est défini dans `.github/workflows/ci.yml` et s'exécute sur :

<https://github.com/GoatsSoftware/GoatUdder>



## 3.2 Étapes du pipeline

1. **Checkout** — Récupération du code source
2. **Setup Node.js** — Installation de Node.js 20
3. **Install dependencies** — `npm ci`
4. **Lint** — ESLint pour la qualité du code
5. **Tests** — Jest pour les tests unitaires
6. **Coverage** — Jest avec couverture ( $\geq 80\%$  branches, functions, lines, statements)
7. **Build Docker** — Build et push vers Harbor
8. **SBOM** — Génération avec Syft
9. **Trivy** — Scan de vulnérabilités CRITICAL
10. **Cosign** — Signature des images Docker
11. **SonarQube** — Analyse de qualité du code



# Chapitre 4

## Rapport de Tests

### 4.1 Tests unitaires (couche Service)

#### 4.1.1 Goat Service

```
goat-service/tests/goat-service.test.js — Tests unitaires du  
GoatService
```

#### 4.1.2 Rental Service

```
rental-service/tests/rental-service.test.js — Tests unitaires du  
RentalService
```

### 4.2 Tests des Controllers (Mocks web avec Supertest)

#### 4.2.1 Goat Controller

```
goat-service/tests/goat-controller.test.js — Tests avec supertest +  
jest.mock()
```

Endpoints testés :

- GET /goats — Retourne toutes les chèvres
- GET /goats/:id — Retourne une chèvre par ID
- GET /goats/available — Retourne les chèvres disponibles
- GET /goats/:id/available — Vérifie la disponibilité d'une chèvre

## 4.2.2 Rental Controller

|                                                             |   |       |      |
|-------------------------------------------------------------|---|-------|------|
| <code>rental-service/tests/rental-controller.test.js</code> | — | Tests | avec |
| <code>supertest + jest.mock()</code>                        |   |       |      |

Endpoints testés :

- GET `/rentals` — Retourne toutes les locations
- GET `/rentals/:id` — Retourne une location par ID
- GET `/rentals/active` — Retourne les locations actives
- POST `/rentals` — Crée une location
- POST `/rentals/:id/complete` — Complète une location

## 4.3 Tests de la couche Data (Repository)

|                                                         |   |       |    |
|---------------------------------------------------------|---|-------|----|
| <code>goat-service/tests/goat-repository.test.js</code> | — | Tests | du |
| <code>GoatRepository</code>                             |   |       |    |

|                                                             |   |                                                |  |
|-------------------------------------------------------------|---|------------------------------------------------|--|
| <code>rental-service/tests/rental-repository.test.js</code> | — | <i>[À ajouter : tests du RentalRepository]</i> |  |
|-------------------------------------------------------------|---|------------------------------------------------|--|

# Chapitre 5

## Rapport de Couverture de Code

### 5.1 Seuils de couverture

Le pipeline CI impose les seuils minimums suivants :

| Métrique   | Seuil Minimum |
|------------|---------------|
| Branches   | $\geq 80\%$   |
| Functions  | $\geq 80\%$   |
| Lines      | $\geq 80\%$   |
| Statements | $\geq 80\%$   |

### 5.2 Résultats de couverture

Les rapports de couverture sont générés par Jest (`lcov`) et uploadés comme artifacts GitHub Actions.



# Chapitre 6

## Rapport de Qualité

### 6.1 SonarQube

L'analyse de qualité est réalisée via SonarQube.

<https://sonarqube.techops.tf/dashboard?id=GoatUdder>

**Note :** Le calcul de la couverture de code effectué par SonarQube diffère de celui réalisé par Jest. Jest mesure la couverture directement à partir des tests exécutés via son rapport de couverture, tandis que SonarQube analyse les rapports importés dans son propre processus d'analyse. La valeur affichée par SonarQube est donc inférieure à celle obtenue avec Jest et ne reflète pas directement le seuil de couverture défini dans le pipeline CI.

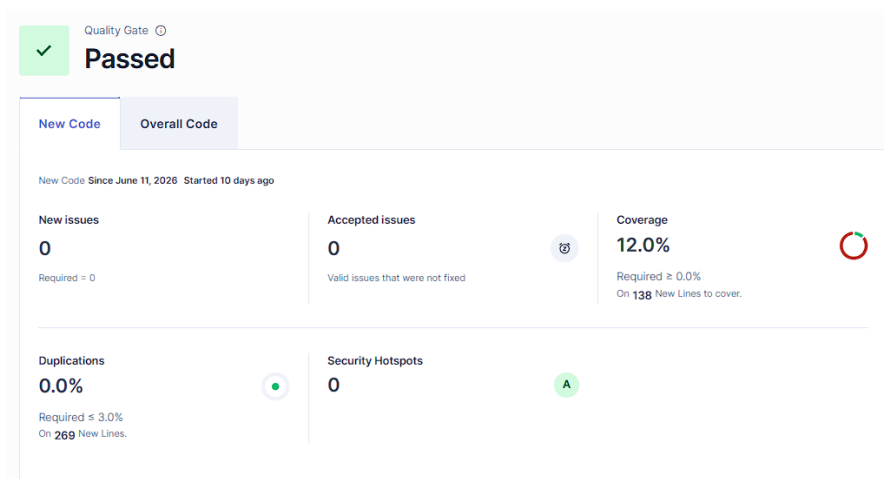


FIGURE 6.1 – Rapport SonarQube pour le projet GoatUdder

## 6.2 ESLint

Le linting est exécuté à chaque build CI pour chaque service :

- goat-service/eslint.config.mjs
- rental-service/eslint.config.mjs

## 6.3 Code coverage de JEST et tests unitaires

Pour 'goat-service' :

| File                           | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s |
|--------------------------------|---------|----------|---------|---------|-------------------|
| All files                      | 96.29   | 97.05    | 89.47   | 96.29   |                   |
| src                            | 86.36   | 75       | 50      | 86.36   | 13,36-37          |
| index.js                       | 86.36   | 75       | 50      | 86.36   |                   |
| src/controller                 | 100     | 100      | 100     | 100     |                   |
| goat-controller.js             | 100     | 100      | 100     | 100     |                   |
| src/data                       | 100     | 100      | 100     | 100     |                   |
| goat-repository.js             | 100     | 100      | 100     | 100     |                   |
| src/service                    | 100     | 100      | 100     | 100     |                   |
| goat-service.js                | 100     | 100      | 100     | 100     |                   |
| Test Suites: 3 passed, 3 total |         |          |         |         |                   |
| Tests: 30 passed, 30 total     |         |          |         |         |                   |
| Snapshots: 0 total             |         |          |         |         |                   |
| Time: 1.721 s                  |         |          |         |         |                   |
| Ran all test suites.           |         |          |         |         |                   |

FIGURE 6.2 – Rapport de couverture de code pour le Goat Service

Pour 'rental-service' :

| File                                                                                                                                      | % Stmts      | % Branch     | % Funcs      | % Lines      | Uncovered Line #s |
|-------------------------------------------------------------------------------------------------------------------------------------------|--------------|--------------|--------------|--------------|-------------------|
| <b>All files</b>                                                                                                                          | <b>91.76</b> | <b>89.65</b> | <b>88.23</b> | <b>91.76</b> |                   |
| src                                                                                                                                       | 84.21        | 75           | 0            | 84.21        | 13,30-31          |
| index.js                                                                                                                                  | 84.21        | 75           | 0            | 84.21        |                   |
| src/controller                                                                                                                            | 87.5         | 80           | 100          | 87.5         | 21,29,38,56       |
| ...l-controller.js                                                                                                                        | 87.5         | 80           | 100          | 87.5         |                   |
| src/service                                                                                                                               | 100          | 100          | 100          | 100          |                   |
| rental-service.js                                                                                                                         | 100          | 100          | 100          | 100          |                   |
| Test Suites: 2 passed, 2 total<br>Tests: 26 passed, 26 total<br>Snapshots: 0 total<br>Time: 0.99 s, estimated 1 s<br>Ran all test suites. |              |              |              |              |                   |

FIGURE 6.3 – Rapport de couverture de code pour le Rental Service

# Chapitre 7

## Captures d'écran des Google Labs

Dashboard Google skills de MENON Valentin

| Activity                                                              | Type     | Date started | Date finished | Score            | Passed |
|-----------------------------------------------------------------------|----------|--------------|---------------|------------------|--------|
| <a href="#">Continuous Delivery with Jenkins in Kubernetes Engine</a> | ▲ Lab    | 10 days ago  | 10 days ago   | Assessment: 100% | ✓      |
| <a href="#">Artifact Registry: Qwik Start</a>                         | ▲ Lab    | 10 days ago  | 10 days ago   | Assessment: 100% | ✓      |
| <a href="#">DevOps Essentials</a>                                     | ■ Course | 10 days ago  |               |                  |        |
| <a href="#">Automating the Deployment of Networks with Terraform</a>  | ▲ Lab    | May 29, 2026 | May 29, 2026  | Assessment: 100% | ✓      |
| <a href="#">Infrastructure as Code with Terraform</a>                 | ▲ Lab    | Apr 24, 2026 | Apr 24, 2026  | Assessment: 100% | ✓      |
| <a href="#">Build Infrastructure with Terraform on Google Cloud</a>   | ■ Course | Apr 24, 2026 |               |                  |        |
| <a href="#">Implementing Cloud Load Balancing for Compute Engine</a>  | ■ Course | Apr 24, 2026 |               |                  |        |
| <a href="#">Getting Started with Google Cloud</a>                     | ■ Path   | Apr 24, 2026 |               |                  |        |
| <a href="#">A Tour of Google Cloud Hands-on Labs</a>                  | ▲ Lab    | Apr 24, 2026 | Apr 24, 2026  | Assessment: 100% | ✓      |
| <a href="#">Create a Virtual Machine</a>                              | ▲ Lab    | Apr 24, 2026 | Apr 24, 2026  | Assessment: 100% | ✓      |

FIGURE 7.1 – Dashboard Google Skills de MENON Valentin

Dashboard Google skills de MASSINOND Matéo

**Progress**

Track your learning activities, pick up where you left off, and celebrate what you've completed

Search by activity name

Course Lab Quiz Game Learning path Classroom [Show All](#)

| Activity                                                             | Type   | Date started | Date finished | Score            |
|----------------------------------------------------------------------|--------|--------------|---------------|------------------|
| <a href="#">Automating the Deployment of Networks with Terraform</a> | Lab    | May 29, 2026 | May 29, 2026  | Assessment: 100% |
| <a href="#">Infrastructure as Code with Terraform</a>                | Lab    | Apr 24, 2026 | Apr 24, 2026  | Assessment: 100% |
| <a href="#">Build Infrastructure with Terraform on Google Cloud</a>  | Course | Apr 24, 2026 |               |                  |
| <a href="#">Create a Virtual Machine</a>                             | Lab    | Apr 24, 2026 | Apr 24, 2026  | Assessment: 100% |
| <a href="#">Getting Started with Google Cloud</a>                    | Path   | Apr 24, 2026 |               |                  |
| <a href="#">A Tour of Google Cloud Hands-on Labs</a>                 | Lab    | Apr 24, 2026 | Apr 24, 2026  | Assessment: 100% |

**Matéo Massinond**  
Member since 2026  
[Starter](#)  
[Buy subscription](#)

[Dashboard](#)  
[Progress](#)  
[Settings](#)  
[Sign Out](#)  
[Privacy](#) [Terms](#)

FIGURE 7.2 – Dashboard Google Skills de MASSINOND Matéo

# Chapitre 8

## Accès aux Ressources

### 8.1 Dépôt Git (GitHub)

Le repository est public et accessible à :

<https://github.com/GoatsSoftware/GoatUdder>

### 8.2 Harbor Registry (Images Docker)

<https://harbor.techops.tf/>

**Credentials :**

— **Username :** bcharroux

— **Password :**

sLdRMfY722FBbvg0e8t6trSy2Efu4NaK8Tr7xRKQyaM8uCQvT3tg98gHyD7cr8iF



# Chapitre 9

## Conclusion

Le projet **GoatUdder** respecte l'ensemble des exigences DevOps :

- ✓ Dépôt Git structuré
- ✓ Pipeline CI avec GitHub Actions
- ✓ Architecture en couches (Data, Service, Controller)
- ✓ 2 services back conteneurisés (Goat Service, Rental Service)
- ✓ Tests de toutes les couches (unitaires, mocks web)
- ✓ Couverture de code  $\geq 80\%$
- ✓ Qualité logicielle (SonarQube, ESLint)
- ✓ Frontend web
- ✓ Base de données PostgreSQL